Privacy-Preserving Sealed-Bid Auctions

CS670 final project presentation

Aarav Oswal (230012) Ch. Rishitha (230311)
R. Charan (230819) Poorvie Sadagopan (230759)

November 2026

CSE, IIT Kanpur

The problem and our setting

The auctioneer's dilemma

A sealed-bid auction needs someone to open every envelope.

- The auctioneer learns every valuation, budget and strategy.
- In a **Vickrey** auction (winner pays the 2nd-highest bid) it can inflate the price undetectably.
- Even an honest auctioneer is a single point of failure.

Question: can we compute the same outcome without *anyone* seeing the bids?

Two auction rules
First-price: highest bidder wins, pays own bid.

Vickrey: highest bidder wins, pays the second-highest bid (truthful bidding is optimal).

An auction is a secure maximum
$i^\star = \arg\max_i b_i, \quad \text{price} = \max_i b_i$
or 2nd max

Goal and threat model

Privacy target
The protocol reveals **the winner and the clearing price, and nothing else**: no ranks, orderings, ties or approximate values of losing bids.

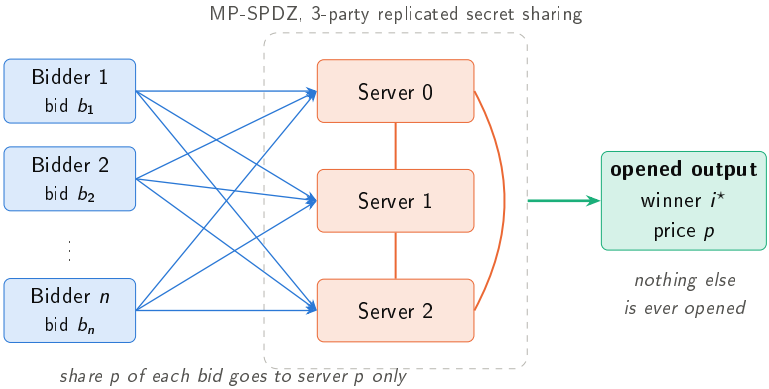
Adversary

- 3 computing servers, at most **1 corrupted**
- **semi-honest**: follows the protocol, tries to learn
- 2 colluding servers: outside the model

What we built

- Kurosawa–Ogata **bit-slice** auction on the **MP-SPDZ** framework
- first-price *and* Vickrey
- arithmetic *and* binary versions
- a comparison-based baseline to measure against

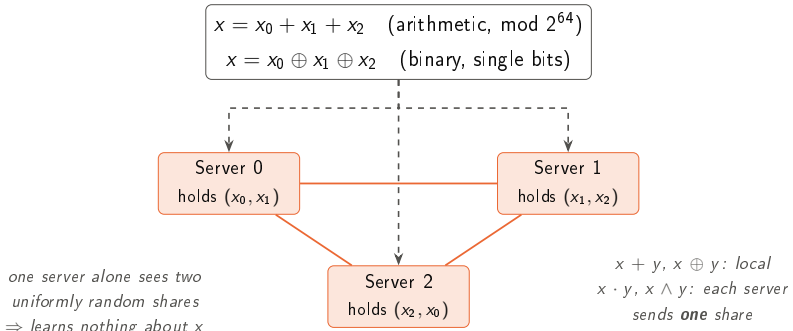
System architecture



Bidders are *not* servers: they send three random-looking shares and leave.

Computing on secret shares

Building block: 3-party replicated secret sharing



Araki et al. (CCS 2016): honest majority, very cheap multiplication.

Adding and multiplying without seeing the values

Add / subtract: each server adds its own shares. No communication.

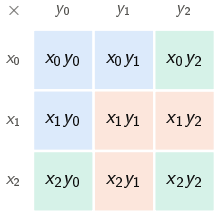
Multiply:

1. each server computes 3 of the 9 cross terms
2. adds a fresh random mask (a sharing of 0)
3. sends **one** number to its neighbour

$\Rightarrow$ one communication round.

On bits: AND $= xy$, NOT $= 1 - x$, OR $= 1 - (1 - x)(1 - y)$

Any algorithm built from these runs on hidden data.



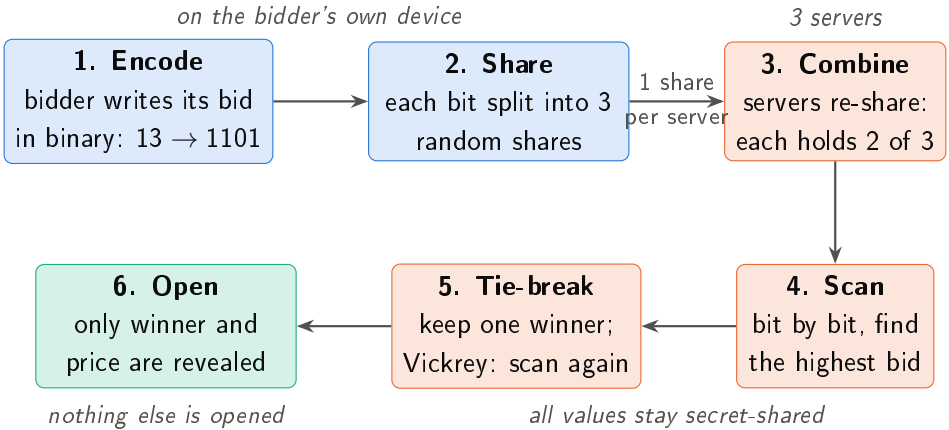
$x \cdot y$ = sum of all 9 squares; each server computes its 3

Server 0 holds x_0, x_1 and y_0, y_1

Server 1 holds x_1, x_2 and y_1, y_2

Server 2 holds x_2, x_0 and y_2, y_0

End-to-end workflow of one auction



Bidder side: sharing the bits of the bid

Bits first, then shares

- the bidder writes its bid in binary
- every bit is split into 3 random shares

$$b = s_0 + s_1 + s_2 \pmod{2^{64}}$$

(s_0, s_1 random; s_2 fixes the sum)

- server p receives only s_p
- binary version: XOR shares, $b = x_0 \oplus x_1 \oplus x_2$

Example: one bit equal to 1

to server 0	8 412 993 001
to server 1	−3 107 551 284
to server 2	−5 305 441 716
sum	1

Why it is safe

One share alone is uniformly random: it looks the same whether the bit is 0 or 1.

The auction algorithm

From pairwise comparisons to a single scan

Naive: compare bids pairwise (a tournament of Millionaires' problems).

Kurosawa–Ogata (2002): scan all n bids at once, one bit position at a time, from the most significant bit down.

- keep a mask S of bidders who can still win
- at bit j: if any survivor has a 1, drop survivors with a 0
- survivors at the end hold the maximum

Only AND and OR on bits are needed.

The catch

Naively: "*if* a survivor has a 1, ..."

A branch on a secret leaks it.

We need a **branch-free** form: the same gates must run whatever the bids are.

Worked example: $A = 11,\ B = 13,\ C = 9$

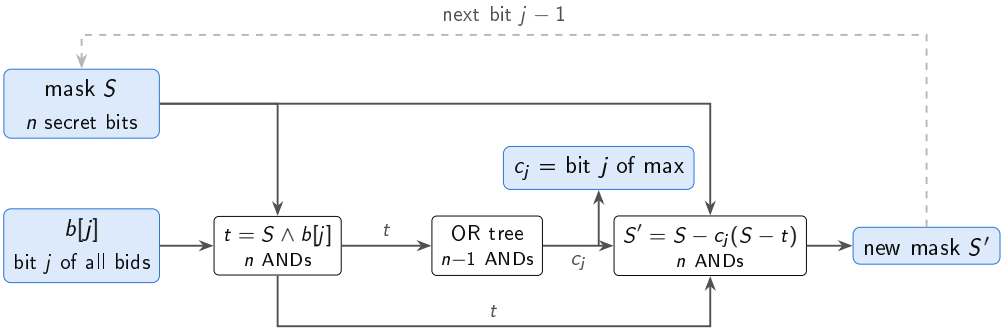
	bit 3 (MSB)	bit 2	bit 1	bit 0 (LSB)
$A = 11$	1	0	1	1
$B = 13$	1	1	0	1
$C = 9$	1	0	0	1
c_j: a survivor has a 1?	1	1	0	1
mask S after this bit	$\{A, B, C\}$	$\{B\}$	$\{B\}$	$\{B\}$

*bit 1: no survivor has a 1 ($c_1 = 0$), so the mask is **carried forward***

max $= 1101_2 = 13$
winner: B

Key case, bit 1: no survivor has a 1, so $c_1 = 0$ and the mask is **kept**, not emptied.

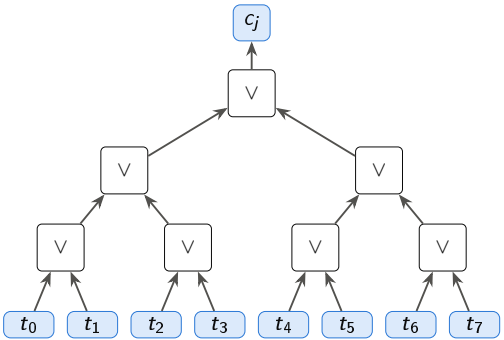
One step of the scan, obliviously



if some survivor has a 1 here ($c_j = 1$), drop the survivors with a 0; otherwise keep S unchanged. No branch: the same gates run either way.

$S' = S - c_j(S-t)$: equals t if $c_j = 1$, stays S if $c_j = 0$ — no branch. Per scan: $k(3n-1)$ ANDs, $k(\lceil \log_2 n \rceil + 2)$ rounds.

The OR tree: “does any survivor have a 1?”



round 3

round 2

round 1

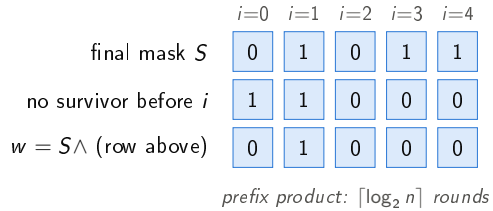
$x \vee y = 1 - (1 - x)(1 - y)$

one AND per gate

$n - 1$ *ANDs,* $\lceil \log_2 n \rceil$ *rounds*

The maximum is free: c_j *is* bit j of the maximum, so $\max = \sum_j c_j 2^j$.

Ties without leaking them

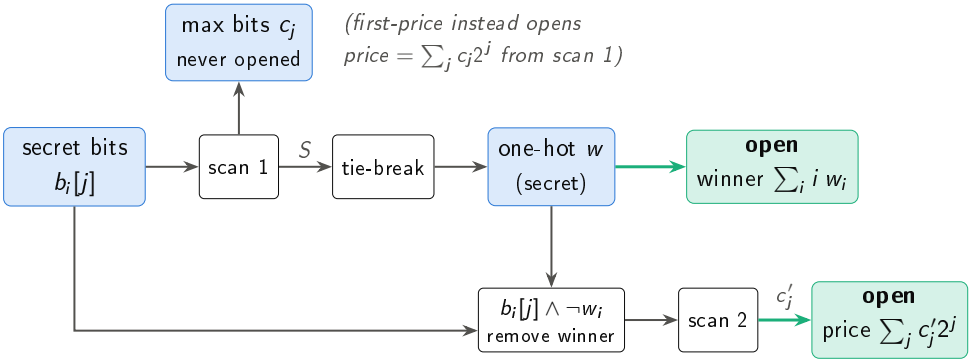


bidders 1, 3, 4 tied; only the index 1 is opened, never the tie

winner $= \sum_i i \cdot w_i = 1$

Opening the final mask would reveal who tied. Instead keep only the first survivor, $w_i = S_i \prod_{l<i}(1 - S_l)$, and open just its index.

Vickrey: a second, oblivious scan



Zero the winner's bits ($b \wedge \neg w$), scan again: the new maximum is the second price. Ties at the top $\Rightarrow$ price = top bid. One bidder $\Rightarrow$ pays 0.

The output: open only winner and price

At the very end the servers combine their shares of exactly **two values**:

- the winner's index $\sum_i i\, w_i$
- the price $\sum_j c_j 2^j$

Never opened: the mask S, the first-scan maximum in Vickrey, any losing bid.

We confirmed with MP-SPDZ's own count of opened values: exactly 2.

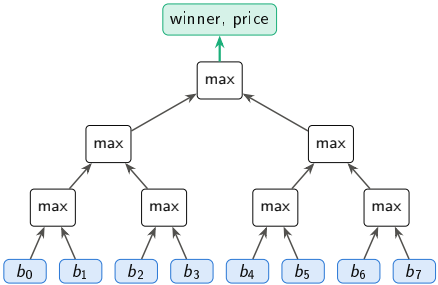
Who sees what

Bidder	own bid + output
One server	random shares + output
Network	encrypted shares
2 servers	outside the model

Implied by the output (not a leak): losing bids $\leq$ price.

Evaluation and conclusions

Baseline: a tournament of secure comparisons



each node: secure comparison $a > b$ + multiplexers on (max, runner-up, index)

$\log_2 n$ *levels* $\times$ $O(\log k)$ *rounds per comparison*

right bracket wins only on strict $> \Rightarrow$ *lowest index wins ties*

- MP-SPDZ's built-in comparison
- same tie rule, same outputs
- built in **both** domains

Same circuit, two domains

Arithmetic domain ($\mathbb{Z}_{2^{64}}$)

one secret bit $b \in \{0,1\}$ is held as:

64-bit share x_0

64-bit share x_1

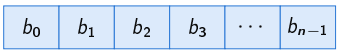
64-bit share x_2

AND = ring multiplication

$\Rightarrow$ ***64 bits*** *sent per server per AND*

Binary domain (XOR-shared bits)

bit j of all n bidders in one word:



lane i = bit j of bidder i (XOR-shared)

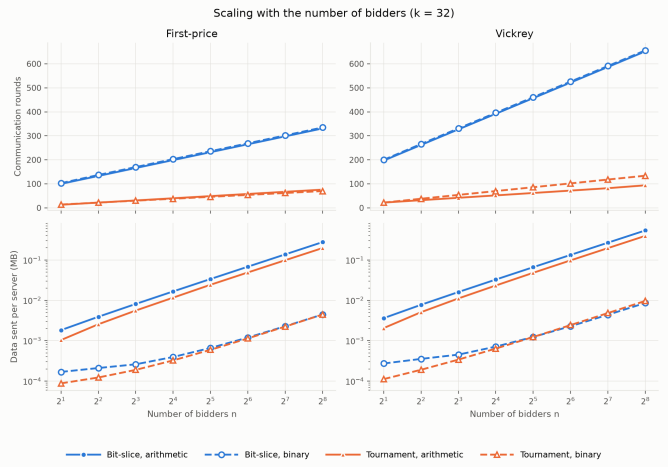
AND of two words = n ANDs in one step

$\Rightarrow$ ***1 bit*** *sent per server per AND*

XOR and NOT are free

same circuit, same AND count (29 505 for n=128, k=32, Vickrey): **0.269 MB** $\rightarrow$ **0.0044 MB** per server (61$\times$ less)

Scaling with the number of bidders ($k = 32$)



3 servers on one machine. The measured AND counts match our formula exactly.

Results at $n = 128$, $k = 32$, Vickrey

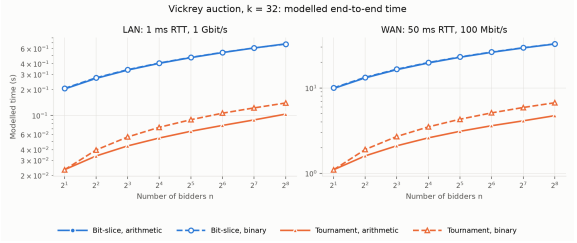
Variant	ANDs	rounds	MB / server	LAN (s)	WAN (s)
Bit-slice, arithmetic	29 505	587	0.269	0.61	29.4
Bit-slice, binary	29 505	591	**0.0044**	0.61	29.6
Tournament, arithmetic	15 621	**82**	0.197	**0.09**	**4.1**
Tournament, binary	34 163	118	0.0049	0.12	5.9

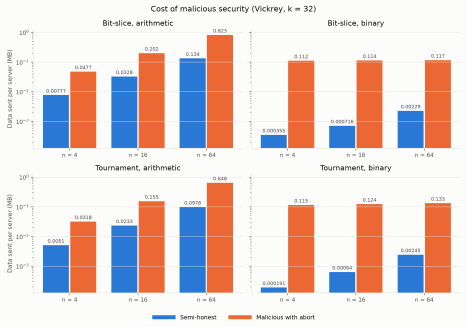
Binary domain: 61× less data
each AND costs 1 bit instead of 64. The baseline gains just as much.

But rounds decide the time
bit-slice handles bits one after another: rounds grow with $k \log n$. The tournament needs far fewer.

LAN: 1 ms, 1 Gbit/s. WAN: 50 ms, 100 Mbit/s. Modelled from measured rounds and bytes.

Modelled network time and the cost of malicious security





Same programs under malicious-security protocols: $\approx 6\times$ data in the arithmetic domain; near-constant $\approx$0.11 MB in the binary domain.

How we checked correctness

Every result is compared with the **plain, non-private auction rule**.

- **The formulas:** simulated on plain bits for every input with up to 3 bidders, plus thousands of random cases with forced ties
- **The real protocol:** three servers run edge cases (ties, one bidder, all zero, maximum bids) and random auctions, also under malicious security
- **The output:** only the winner and the price are opened
- **The benchmarks:** every run's result checked as well

What we learned

- **The domain matters more than the algorithm.** Binary vs. arithmetic: 61× communication; bit-slice vs. tournament: about 10%.
- **Bit-slice loses on latency.** Its scan is sequential in the bid length; modern comparisons need only $O(\log k)$ rounds. Kurosawa–Ogata's "7× faster" was measured against 2002-era comparisons.
- **Where bit-slice still wins:** fewest ANDs at large n (binary), exactly 2 opened values, only multiplication needed, and it ports to server-free designs (SEAL).
- **Secret data forces branch-free algorithms.** Every "if" on a secret becomes arithmetic, so the same operations run whatever the bids are.

Limitations and future work

Limitations

- bidders are trusted to send valid bits
- a malicious server could swap its own input share
- all servers on one machine; network times modelled

Future work

- authenticated bidder inputs
- public verifiability: commitments + ZK proofs
- radix-2^r scan: $r\times$ fewer rounds
- more servers (Shamir); SEAL (no servers)

Summary

Three servers compute a first-price or Vickrey auction without any of them seeing a bid, and open only the winner and the price.

Thank you!

Questions?

github.com/rcharan05/CS670_Project